

1 Objective

In this group assignment, you will build a *retrieval-augmented generation* (RAG) system for a natural language processing task of your choice. You are free to pick any downstream task that retrieved evidence can serve, e.g., open-domain question answering, summarization, fact-checking, semantic search, recommendation, or sentiment and opinion analysis (for instance, a system that finds relevant opinions about any instance of a topic of your choice, such as bitcoin within the topic of cryptocurrencies, and reports that they are 70% positive and 30% negative). Whatever task you choose, the heart of your system must be the retriever-generator contract of this course: a knowledge base that is gathered and indexed offline, a retriever that locates the most relevant evidence for a query, and a generator (or classifier) that produces the final answer conditioned on that evidence. Once you have chosen a task and a domain, make sure that a) you can gather enough data about it (e.g., some topics may be too niche to the point that you would only find a few hundreds documents about it) and b) the data are suitable and balanced for the task (e.g., for sentiment analysis, if the topic you chose only has negative opinions associated with it, it's probably not a good topic). For ideas about interesting topics, you can check our project page at <https://sentic.net/projects>. You are pretty much free to use anything you want in terms of available tools/libraries. However, your system cannot be just a mashup of existing services or a single call to a hosted RAG API: you must implement, and be able to explain, the retrieval and generation stages yourselves. Your final score will depend not only on how you developed your system but also on its novelty and your creativity: in other words, to get a high score you do not only need to implement a system that works, but also a system that is useful and user-friendly.

2 Deadline and Grouping

The assignment constitutes 35% of your total grade for the course. Assignments are to be submitted via Blackboard (email submissions will not be considered) by **7th November at 11:59pm SGT**. 5% points will be deducted for each rounded-off day after the deadline. Only the first submission counts (Blackboard allows for multiple submissions only in case of system errors or disconnections). The assignment will be done in groups of 6 or 5 people. Members will be randomly assigned by the system for fairness. The main tasks of the assignment are knowledge-base construction (20 points), retrieval (40 points) and the downstream task with generation (40 points). If you like, you can split your group into up to three subgroups taking care of each of these tasks and specify who did what in your final report so that each member will be graded accordingly. If this information is not specified, a unique grade will be given to the whole project and this will be shared among all members of the group (recommended option). Some overlap between projects from different groups is allowed but beware that, if we find out that a project has more than 30% overlap with another project from this year or past years, your group will be disqualified (and get zero points as final grade for the assignment). Hence, it is OK to share the general idea of your assignment with other groups but not the implementation details.

3 Tasks and Questions

3.1 Knowledge-Base Construction (20 points)

Assemble the corpus that your system will retrieve from. Gather text data from any sources which you are interested in and permitted to access, e.g., [Reddit API](#) or a public dataset, and/or combine them with ready-made datasets. The corpus should have at least 10,000 documents (or passages) and at least 100,000 words. It is OK to use available datasets for training or as background knowledge (e.g., [popular sentiment benchmarks](#) or datasets from the [Hugging Face Hub](#)), but you still have to at least gather and label a held-out test set yourselves. For your own evaluation dataset, make sure it does not contain duplicates and try your best to make it balanced (e.g., an equal number of positive and negative entries for sentiment). For a classification task, use the same tabular format as the above-mentioned sentiment benchmarks and name it “eval.xls” (using a different format will result in demerit points); for other tasks, use an appropriate standard format (e.g., JSONL) and document it. Because a generator will read what you retrieve, you must also clean and *chunk* the raw documents into passages of a sensible size, which is itself a design choice discussed in the course. You can use any third-party libraries for gathering and preparing the data, e.g.:

- Hugging Face Datasets: <https://huggingface.co/datasets>
- Kaggle Datasets: <https://www.kaggle.com/datasets>
- Common Crawl: <https://commoncrawl.org>
- Wikimedia dumps: <https://dumps.wikimedia.org>
- Scrapy: <https://scrapy.org>
- BeautifulSoup: <https://www.crummy.com/software/BeautifulSoup>
- Trafilatura: <https://trafilatura.readthedocs.io>
- Playwright: <https://playwright.dev>
- PRAW (Reddit): <https://praw.readthedocs.io>

Note that access to social-media data has tightened in recent years: the X (formerly Twitter) API now requires a paid tier and several platforms have restricted their public APIs, so prefer open datasets and permissively licensed sources, and always respect each source's terms of use and rate limits.

Question 1: Explain and provide the following:

1. How you gathered the corpus (e.g., source, keywords, API, library), and how you cleaned, chunked, and stored it
2. Which task users will perform over your corpus (i.e., the application), with sample queries or inputs
3. The numbers of documents (and chunks), words, and types (i.e., unique words) in the corpus

3.2 Retrieval (40 points)

Retrieval: Build a retriever over your knowledge base. At a minimum, implement both a *sparse* (lexical) retriever and a *dense* (embedding) retriever, and combine them into a *hybrid* pipeline, optionally followed by a re-ranker, exactly the design developed in this course. You can build each stage from scratch or use some combination of available tools. Useful components include:

- Elasticsearch / OpenSearch (inverted index, BM25): <https://www.elastic.co>, <https://opensearch.org>
- Apache Solr / Lucene: <https://solr.apache.org>, <https://lucene.apache.org>
- Pyserini (reproducible BM25 and dense retrieval): <https://github.com/castorini/pyserini>
- Sentence-Transformers (SBERT embeddings): <https://sbert.net>
- Hugging Face Transformers (BGE, GTE, E5 encoders): <https://huggingface.co/models>
- FAISS (similarity search library): <https://github.com/facebookresearch/faiss>
- Vector databases: [Chroma](#), [Qdrant](#), [Weaviate](#), [Milvus](#), [pgvector](#)
- RAG orchestration frameworks: [LangChain](#), [LlamaIndex](#), [Haystack](#), [DSPy](#)

You can also choose any other inverted-index or vector-search open project. However, you should NOT simply adopt SQL-based solutions for text search (for example, you CANNOT solve retrieval simply using a LIKE query in Microsoft SQL Server or MySQL).

Querying: You need to provide a simple but friendly user interface (UI) for querying. It could be either a web-based or mobile app based UI. Lightweight options such as [Streamlit](#) or [Gradio](#) let you build one in a few lines; [FastAPI](#) or [Django](#) are options for a fuller web app. The UI must be kept simple. A sophisticated UI is not necessary nor encouraged (as it is not the focus of this course). Detailed information besides text is allowed to be shown for the query results, e.g., images, ratings, timestamps, and source links. The details should be designed to solve specific problems.

Question 2: Perform the following tasks:

- Design a simple UI (you can design one from scratch or you can tap on an existing one) to allow users to query your system in a simple way
- Write five queries, get their results, and measure the speed of the querying

Question 3: Explore some innovations for enhancing the retrieval and ranking. Explain why they are important to solve specific problems, illustrated with examples. You can list anything that has helped improving your system from the first version to the last one, plus queries that did not work earlier but now work because of the improvements you made. Because retrieval sets a ceiling on everything downstream, evaluate it explicitly with rank-aware measures such as Precision@K, Recall@K, Mean Reciprocal Rank, and Normalized Discounted Cumulative Gain (e.g., using ir_measures, BEIR, or MTEB for embedding quality). Possible innovations include (but are not limited to) the following:

- Hybrid fusion (e.g., reciprocal rank fusion of sparse and dense results)
- Re-ranking (e.g., a cross-encoder or a learned re-ranker over the top candidates)
- Query expansion or rewriting (e.g., pseudo-relevance feedback, or HyDE-style hypothetical documents)
- Timeline and metadata search (e.g., allow users to search within specific time windows or categories)
- Multimodal search (e.g., implement image or table retrieval alongside text)
- Multilingual search (e.g., enable your system to retrieve data in multiple languages)
- Chunking and indexing strategies (e.g., semantic chunking or parent-document retrieval)

3.3 Downstream Task and Generation (40 points)

Use the retrieved evidence to solve your chosen task. For a *generative* task (question answering, summarization, fact-checking), condition a language model on the retrieved passages, so that the answer is grounded in evidence a user can inspect rather than produced from the model's parameters alone. For a *classification* task such as sentiment analysis, which is actually a complex suitcase research problem of many subtasks, you may feed the retrieved evidence to a language model as context or use it as features for a dedicated classifier; unless you are sure that your data does not contain neutral content, you should cover at least *subjectivity detection* and *polarity detection* (first *neutral* versus *opinionated*, then *positive* versus *negative*). Different approaches can be applied, including:

- knowledge based, e.g., SenticNet <https://sentic.net>
- rule based, e.g., linguistic patterns
- machine learning based, e.g., fine-tuned transformers or deep neural networks
- hybrid (a combination of any of the above, i.e., symbolic and subsymbolic AI)

You can tap into any resource or toolkit you like, as long as you motivate your choices and you are able to critically analyze obtained results. Some possible choices include:

- Hugging Face Transformers: <https://github.com/huggingface/transformers>
- PyTorch: <https://pytorch.org>
- TensorFlow / Keras: <https://www.tensorflow.org>, <https://keras.io>
- SciKit-learn: <https://scikit-learn.org>
- spaCy: <https://spacy.io>
- NLTK: <https://www.nltk.org>
- Local LLM serving: Ollama, vLLM, llama.cpp
- Open-weight model families (on the Hugging Face Hub): Llama, Mistral, Qwen, Gemma, Phi, DeepSeek
- Hosted LLM APIs (optional): OpenAI, Anthropic, Google, Cohere

Question 4: Perform the following tasks:

- Motivate the choice of your generation or classification approach in relation with the state of the art
- Discuss whether you had to preprocess data (e.g., microtext normalization) and why
- Build an evaluation dataset by manually labeling at least 1,000 records with an inter-annotator agreement of at least 80% (it is recommended to have 3 annotators, but 2 is also OK)
- Provide task-appropriate quality metrics on such dataset: precision, recall, and F-measure for classification; Exact Match and F1 for extractive QA; ROUGE or BERTScore for summarization
- Provide answer-level RAG metrics, i.e., faithfulness (is every claim grounded in the retrieved passages?), answer relevance, and context relevance/precision, e.g., using RAGAS
- Discuss performance metrics, e.g., latency, records processed per second, cost, and scalability of the system

Question 5: Explore some innovations for enhancing the downstream task. If you introduce more than one, perform an ablation study to show the contribution of each innovation. For example, if you add re-ranking and query rewriting, show the gain from adding only re-ranking, the gain from adding only query rewriting, and the gain from adding both. Explain why they are important to solve specific problems, illustrated with examples. Possible innovations include (but are not limited to) the following:

- Better grounding (e.g., cite sources, or add a faithfulness/self-check step that flags unsupported claims)
- Advanced prompting or reasoning (e.g., chain-of-thought, self-consistency)
- Iterative or agentic RAG (e.g., let the system issue follow-up retrievals until it has enough evidence)
- Fine-tuning or instruction-tuning the generator on your task
- Enhanced classification (e.g., add a subtask such as sarcasm detection, or perform two tasks jointly)
- Fine-grained classification (e.g., perform aspect-based sentiment analysis)

4 Submission

Select one person of the group in charge of submitting the final assignment report. Submission has to be done via Blackboard (do not email your report). As you write your report, it is strongly suggested that you also prepare some slides along with it because your work will also have to be presented in person on week 13. **The submission shall consist of one single PDF file named after your group number**, e.g., if you are group 10, your file should be titled simply “10.pdf”. Failing to name the file correctly or sending it in the wrong format, e.g., zip or MS Word, will result in demerit points. Do add some pictures to your report to make it clearer and easier to read. There is no page limit and no special formatting is required. The file shall contain the following five key items:

1. The names of all group members in the first page
2. The matriculation number of all group members in the first page
3. Your answers to all the above questions
4. A Dropbox (or similar, e.g., Google Drive or OneDrive) link to a compressed (e.g., zip) file with your knowledge base, queries and their retrieved results, evaluation dataset, generated answers or classification results, and any other data for Questions 3 and 5
5. A Dropbox (or similar, e.g., Google Drive or OneDrive) link to a compressed (e.g., zip) file with all your source codes and libraries, with a README file that explains how to compile and run the source codes